KẾ HOẠCH KIẾN TRÚC · ANALYTICS · BẢN THỐNG NHẤT ĐỂ LÀM

Kế hoạch Rollup Analytics

Chuyển các tab phân tích từ "tính lúc có người xem trên bảng thô" sang "tính sẵn theo ngày, đọc nhanh", giữ nguyên số liệu, không đụng `gohub_dw`, không phát sinh chi phí. Thứ tự làm: Quarter Report, B2B Performance, B2C Performance, Channels, rồi các tab còn lại.

Tài liệu này là chuẩn tên gọi, mô hình dữ liệu và quy trình cho mọi phiên làm việc sau. Bản gốc trong repo: `docs/plans/analytics-rollup.md`.

Điều đã chốt

Bốn ràng buộc và bốn quyết định của Hiếu. Mọi thiết kế dưới đây phải thỏa cả tám.

Ràng buộc

R1 `gohub_dw` chỉ đọc. Không sửa, không thêm bảng, chỉ mục, view. Chỉ chạy SELECT.

R2 Không phát sinh chi phí: chỉ dùng gói free đang có (Turso free, Vercel Hobby, cron-job.org).

R3 Quy mô tương lai suy ra từ dữ liệu thật bằng kịch bản, không dựa vào con số Hiếu đưa.

R4 Số liệu các tab không đổi. Mỗi tab chỉ được bật rollup sau khi đối chiếu khớp bảng thô.

Quyết định

Q1 Nơi lưu: Turso gói free, database riêng `intel-rollup`.

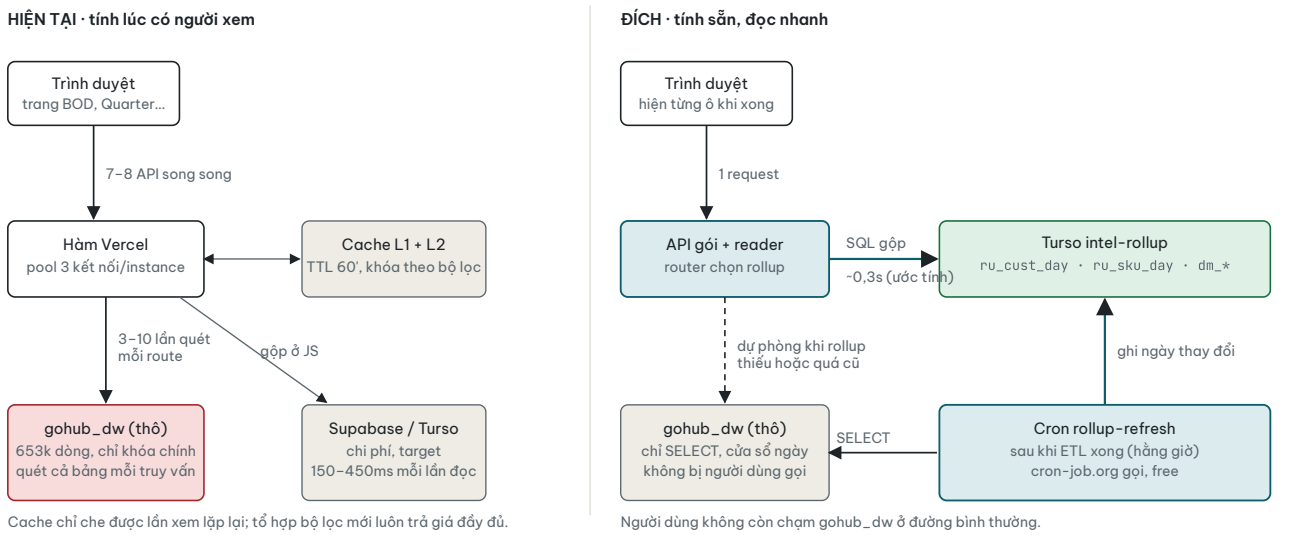
Q2 Số đơn: lưu theo từng ngày, chấp nhận lệch nhỏ khi đơn giao nhiều ngày (đo thật: tối đa 0,9%).

Q3 Thứ tự tab: Quarter Report, B2B Performance, B2C Performance, Channels, các tab khác.

Q4 Độ trễ dữ liệu: theo chu kỳ ETL hằng giờ, như hiện tại.

Hiện trạng và đích

Điểm khác nhau duy nhất cần nhìn: truy vấn nặng chuyển từ đường của người dùng sang đường của cron.



Cái đổi duy nhất: người dùng đọc Turso (nhỏ, có chỉ mục theo ngày), còn gohub_dw chỉ bị cron đọc mỗi giờ một lần với cửa sổ ngày hẹp. Nét đứt là đường thô cũ, giữ làm dự phòng và để đối chiếu.

Số đo thật làm nền cho thiết kế

Đo ngày 28/10/2026 bằng SELECT đọc-only qua endpoint có sẵn. Mọi con số "ước tính" ở các phần sau ghi rõ là ước tính.

652.741
dòng `fact_fulfillment_revenue` (186 MB, 01/2025 đến 10/2026)

≈ 410 MB
cả kho `gohub_dw`, vừa trong bộ nhớ của bất kỳ máy nào

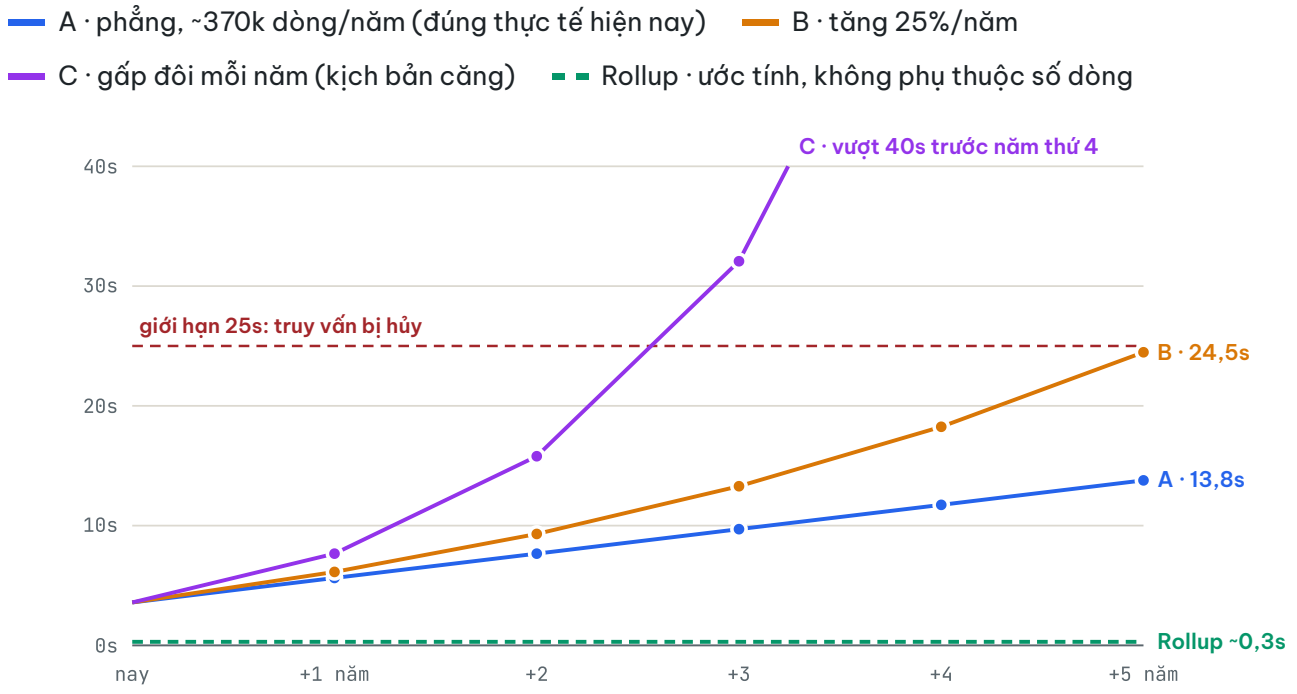
+1%
dòng 9 tháng đầu 2026 so với cùng kỳ 2025 (244.740 so với 242.329)

2,3–5,7 µs
mỗi dòng của cả bảng cho truy vấn kiểu BOD (1,5–3,7s lần đầu)

PHÁT HIỆN	SỐ ĐO	HỆ QUẢ CHO THIẾT KẾ
Chỉ mục	chỉ khóa chính `id`; cột ngày là `varchar`	mọi truy vấn quét cả bảng, xem "tháng này" tốn bằng xem "cả năm"
Quét thuần	`COUNT(*)` ≈ 0,35s lần đầu (đã trừ ~0,3s mạng)	nút thắt là xử lý từng dòng (nhóm, JOIN, `DISTINCT`), không phải quét
Ép kiểu ngày	786ms (ép kiểu) so với 611ms (so chuỗi) lần đầu	đổi cách viết điều kiện ngày không đủ để cứu
Số dòng sau khi gộp	ngày×kênh×công ty×khách×ship: **205.376**; ngày×kênh×công ty×SKU: **258.353**; ngày×kênh×khách×SKU: 373.598	chọn hai rollup đầu; rollup khách×SKU chỉ giảm 43%, bỏ
Đơn nhiều ngày giao	4.546 trên 499.003 đơn (0,91%)	cộng số đơn theo ngày chỉ lệch tối đa ~0,9%
Lịch ETL	dim `:40`, COGS `:45`, fact giao hàng `:50` hằng giờ	làm mới rollup sau `:50`, theo `end_time` của job
Khách trong fact	137.947 mã khách, 7.791 SKU, 76 nguồn, 3 công ty	khách B2C cũng ở cấp từng người, rollup khách phải chứa đủ

Dự đoán quy mô: truy vấn thô sẽ chậm đến đâu

Thời gian một truy vấn nặng lúc cache nguội = số dòng × 5,5 µs (cận trên đo được). Giới hạn của app là 25s. Ba kịch bản thay vì một con số, vì mới có một chu kỳ mùa vụ.



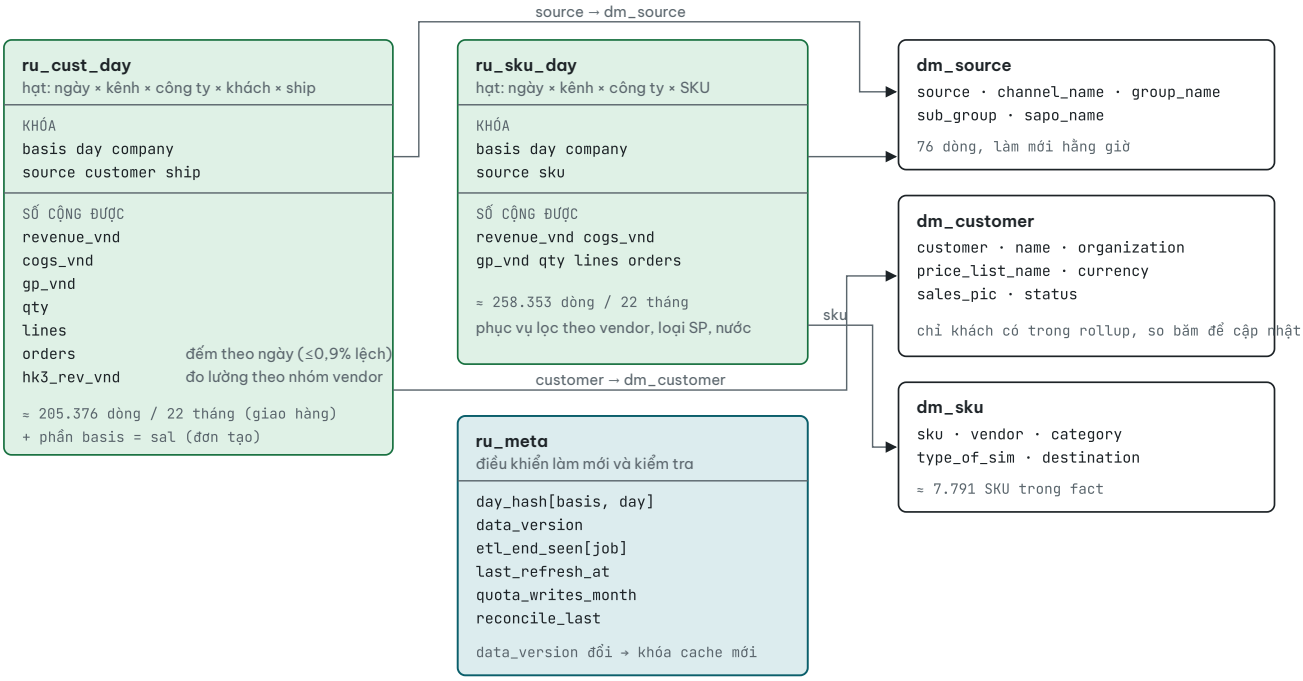
Cách đọc: mỗi chấm là thời gian của MỘT truy vấn nặng khi cache nguội; một tab chạy 3 đến 5 truy vấn nên chờ nhân lên tương ứng. Kịch bản C vỡ trong khoảng năm thứ 3, kịch bản B sát giới hạn ở năm thứ 5. Đường rollup nằm ngang vì mỗi request chỉ đọc một khoảng ngày có chỉ mục (số dòng đọc không tăng theo lịch sử).

KỊCH BẢN	HIỆN TẠI	+1 NĂM	+3 NĂM	+5 NĂM
A · phẳng	0,65M · 3,6s	1,02M · 5,6s	1,76M · 9,7s	2,50M · 13,8s
B · +25%/năm	0,65M · 3,6s	1,12M · 6,1s	2,42M · 13,3s	4,45M · 24,5s
C · ×2/năm	0,65M · 3,6s	1,39M · 7,7s	5,83M · 32,1s	23,6M · 129,8s

giả định Số dòng thêm mỗi năm ở kịch bản A lấy từ 12 tháng gần nhất (~370k). Hệ số 5,5 µs/dòng là cận trên của 3 truy vấn đã đo (2,3 đến 5,7). Q4/2025 cao gấp 1,7 lần trung bình nhưng Q4/2026 chưa lặp lại, nên không dùng mùa vụ để ngoại suy.

Mô hình dữ liệu trong Turso

Hai bảng rollup lưu **mã** và **số cộng được**; mọi nhãn (tên kênh, tên khách, vendor) lấy từ bản sao bảng dim lúc đọc, nên đổi tên không phải dựng lại rollup.

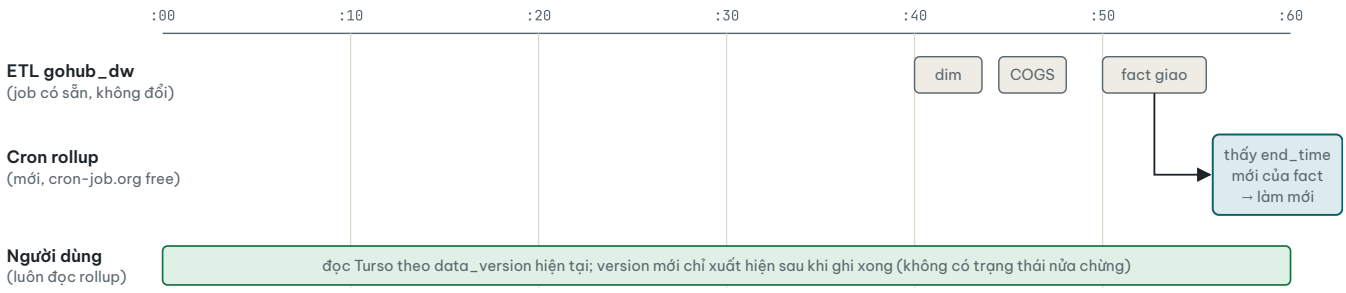


Nối bằng mã lúc đọc, trong cùng một truy vấn SQLite. Không lưu nhãn trong bảng rollup.

Vì sao hai bảng, không phải một: rollup khách×SKU chỉ nhỏ hơn bảng thô 43% (373.598 so với 652.741 dòng), không giải được bài toán quy mô. Tách hai hạt giúp mỗi bảng nhỏ hơn bảng thô 2,5 đến 3,2 lần, và phần lớn truy vấn cần khách (loại trừ khách, phân hạng tier, nhân viên phụ trách) hoặc cần SKU (vendor, nước), hiếm khi cần cả hai.

Quy trình làm mới hằng giờ

Cron chỉ ghi lại ngày có băm khác, để ngân sách ghi của Turso free không bị đốt.



Giờ chạy cụ thể của cron lấy từ `end_time` của job ETL (đã có sẵn trong `etl-cache-sync`), không đặt cứng một phút nào. Nếu ETL chạy trễ, rollup chờ theo.

Mỗi giờ (sau ETL)

1. Đọc cửa sổ 45 ngày gần nhất từ `gohub_dw` (điều kiện ngày so chuỗi, nên chi phí chủ yếu là quét, không xử lý cả bảng).

2. Tính băm theo (basis, ngày), so với `ru_meta.day_hash`.

3. Ghi lại CHỈ ngày đổi: xóa ngày đó rồi chèn lại, trong một giao dịch.

4. Tăng `data_version`; khóa cache của route chứa version nên tự làm mới, không cần xóa hàng loạt.

Mỗi đêm

Đối chiếu tổng theo từng tháng giữa rollup và bảng thô (chia theo tháng để không chạm giới hạn 25s). Tháng lệch được dựng lại. Bắt trường hợp ETL sửa số cũ, ví dụ cập nhật COGS.

`báo lỗi` lệch quá ngưỡng thì báo qua `alertCronFailure` sẵn có.

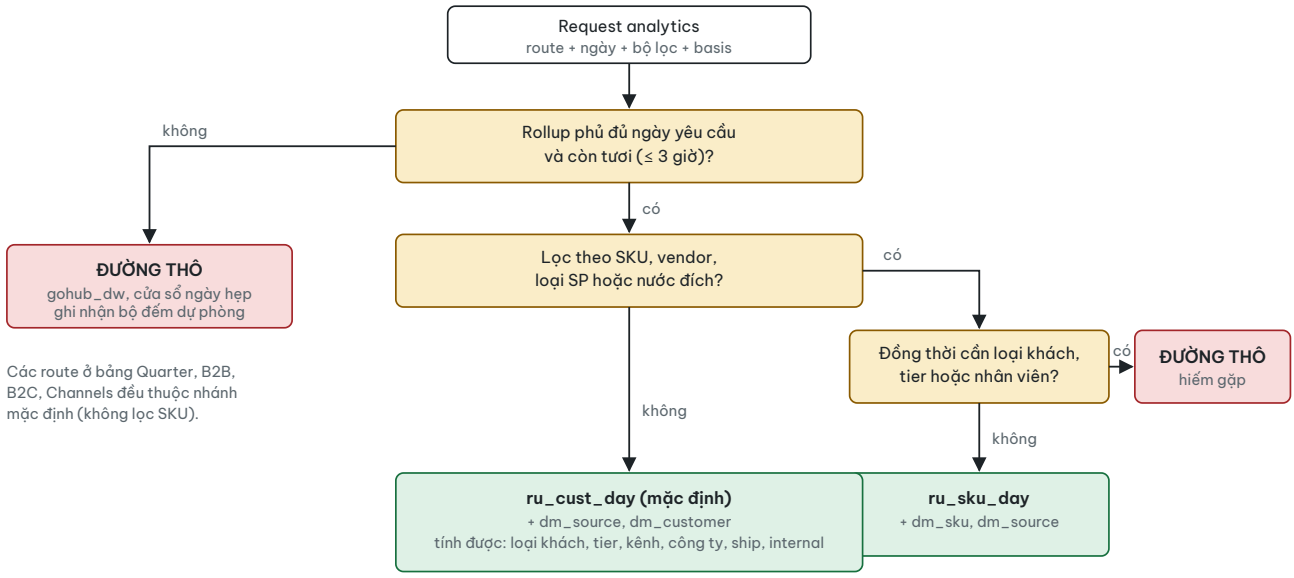
Mỗi tuần

Làm mới đầy đủ các bản sao `dm_*` theo băm; dựng lại các số đo theo nhóm vendor (ví dụ `hk3_rev_vnd`) nếu bảng vendor đổi.

Kiểm hạn mức Turso: ghi, đọc, dung lượng. Quá 70% thì giảm cửa sổ làm mới về "hôm nay và hôm qua".

Router: một request đi vào bảng nào

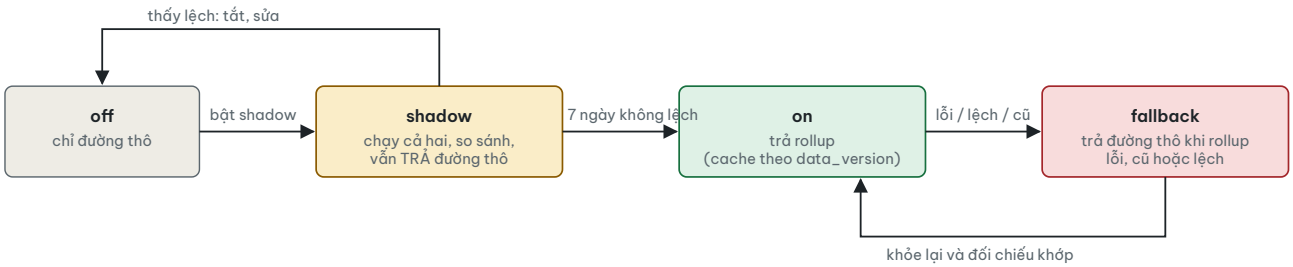
Cùng ý với "aggregate awareness" của Looker: chọn bảng nhỏ nhất đủ trả lời, rơi về bảng thô khi không đủ.



Luật chọn: luôn thử rollup trước; thiếu dữ liệu hoặc quá cũ thì rơi về bảng thô. Tổ hợp "lọc SKU và cần loại khách" rất hiếm (ví dụ BOD khi chọn vendor); ghi bộ đếm để quyết sau có đáng dựng thêm bảng khách×SKU hay không.

Mỗi tab đi qua ba trạng thái trước khi bật hẳn

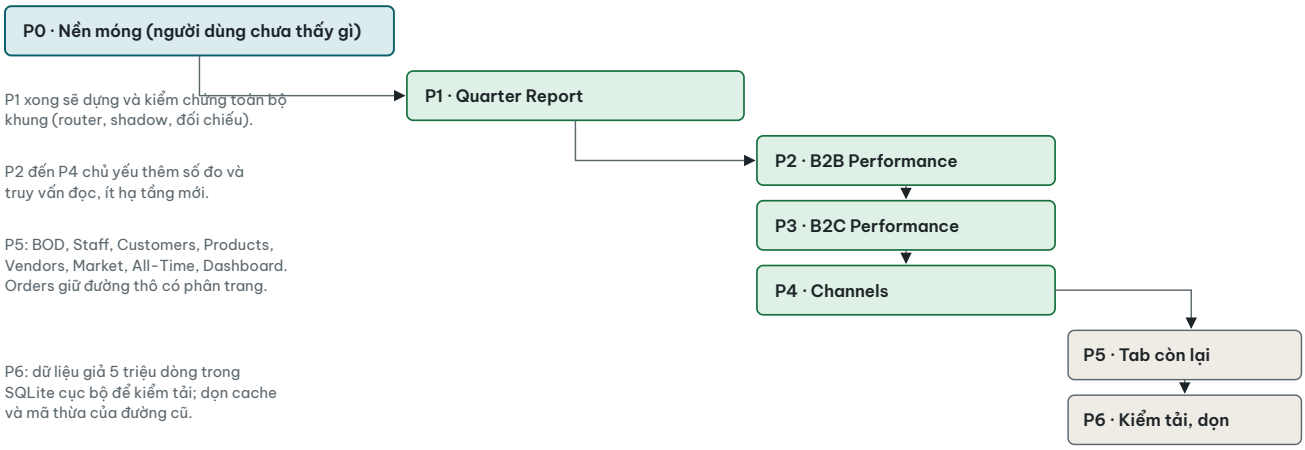
Bật theo từng route, đảo ngược được bằng một cờ trong `app_settings`, không cần deploy.



Cổng chuyển sang on: đối chiếu khớp trong mọi tháng và mọi nhóm, 7 ngày shadow liên tục (khoảng 168 lần làm mới), thời gian lạnh theo mục tiêu bên dưới. Khớp nghĩa là doanh thu, giá vốn, GP, số lượng bằng nhau; số đơn lệch tối đa 1%.

Lộ trình theo tab

Làm tuần tự, tab sau dùng lại phần đã dựng của tab trước. Thứ tự là cam kết; thời gian chưa ước lượng vì chưa đo công việc audit.



Mỗi nhánh chỉ bắt đầu sau khi giai đoạn trước qua cổng (bảng dưới). Quarter Report đi đầu vì có số truy vấn nhiều nhất và đã có sẵn bộ kiểm tra nhất quán giữa các tab.

GIAI ĐOẠN	VIỆC CHÍNH	CỔNG ĐỂ SANG GIAI ĐOẠN SAU
P0 Nền móng	Tạo DB `intel-rollup`; schema; dựng 22 tháng cho cả hai cơ sở (giao hàng, đơn tạo); bản sao dim; cron làm mới; đối chiếu; trang `rollup-health` trong Data Map; giám sát hạn mức	đối chiếu 0 lệch mọi tháng; một lần làm mới ≤ 30s; ghi/đọc trong hạn mức
P1 Quarter Report	audit SQL các route của tab; thêm số đo cần thiết (nhóm vendor); reader + router; shadow rồi on	7 ngày shadow không lệch; lạnh ≤ 1,5s, nóng ≤ 0,5s (mục tiêu)
P2 B2B Performance	`b2b/performance`, `trend`, `kpis`, `tier-performance`, `strategic-performance`, `ecom-*`	như P1
P3 B2C Performance	`b2c/performance`, `kpis`, `trend`, `monthly`, `metric`; `loss-skus` dùng `ru_sku_day`	như P1
P4 Channels	`channels/performance`, `kpis`, `trend`, `b2b-customers`	như P1
P5 Tab còn lại	BOD (`bod-*`), Staff, Customers, Products, Vendors, Market, All-Time, Dashboard, Fulfillment; Orders giữ đường thô	như P1, từng tab một
P6 Kiểm tải và dọn	tạo dữ liệu giả 5M dòng trong SQLite để kiểm thời gian đọc; xóa cơ chế cache thừa; cập nhật wiki	đọc rollup ở 5M dòng giả vẫn trong mục tiêu

Tab nào cần rollup nào

`cần audit` Bảng này lấy từ việc đọc cấu trúc SQL của từng route, chưa đọc hết từng điều kiện. Bước đầu của mỗi giai đoạn là audit để chốt chính xác số đo và phép lọc.

TAB	ROUTE ĐÃ THẤY	ROLLUP	LƯU Ý ĐÃ BIẾT
Quarter Report	`quarterly-report`, `quarterly-b2b-customers`, `quarterly-daily`, `squad-progress`, `quarterly-customer-lifecycle`	`ru_cust_day` + `dm_customer`, `dm_staff`	cần `hk3_rev_vnd`; lifecycle (lần mua đầu/cuối) chỉ thấy từ 01/2025 như bảng thô, phải khớp kết quả hiện tại
B2B Performance	`b2b/performance`, `trend`, `kpis`, `tier-performance`, `strategic-performance`, `ecom-*`	`ru_cust_day`	nhóm theo khách, nhân viên, kênh; phân hạng tier theo `price_list_name` tính lúc đọc
B2C Performance	`b2c/performance`, `kpis`, `trend`, `monthly`, `metric`, `loss-skus`	`ru_cust_day`; `loss-skus` dùng `ru_sku_day`	danh sách loại trừ của B2C khác B2B (`excludedForB2C`)
Channels	`channels/performance`, `kpis`, `trend`, `b2b-customers`	`ru_cust_day`	nhiều truy vấn `COUNT(DISTINCT order_code)`: dùng `orders` theo ngày (Q2)
BOD	`bod-summary`, `-report`, `-group-margin`, `-channel-performance`, `-b2b-tiers`	`ru_cust_day`; khi lọc vendor: `ru_sku_day` hoặc đường thô	gồm cả kỳ trước và cùng kỳ năm ngoái (so sánh 3 kỳ)
Products, Vendors, Market, Region	`products`, `vendors`, `market`, `region-chart`	`ru_sku_day` + `dm_sku`	nước đích suy từ mã SKU (`getDestinationSQL`), tính vào `dm_sku`
Orders	`order-report`, `recent-orders`	không dùng rollup	cần dòng cấp đơn; giữ đường thô với điều kiện ngày hẹp và phân trang

Ngân sách miễn phí

`ước tính` Hạn mức Turso free hiện hành: 5 GB lưu trữ, 500 triệu lượt đọc/tháng, 10 triệu lượt ghi/tháng (trang giá Turso). Phần dùng bên dưới là suy ra từ số dòng đã đo, sẽ thay bằng số đo thật ở P0.

Ghi mỗi tháng		~1,0M / 10M (10%)
Đọc mỗi tháng		~30M / 500M (6%)
Dung lượng		~70 MB / 5 GB (1,4%)

Cách ước tính. Ghi: mỗi giờ thường chỉ 1 đến 2 ngày đổi, khoảng 720 dòng mỗi ngày ở hai bảng, cộng 0,7M dòng dựng lần đầu. Đọc: 200 lượt xem lạnh mỗi ngày × 5 truy vấn × ~30k dòng quét. Dung lượng: ~0,7M dòng × ~100 byte. **Chốt chặn:** nếu ghi vượt 70% hạn mức, cron tự giảm cửa sổ làm mới về hôm nay và hôm qua; vượt 90% thì dừng ghi và mọi route rơi về đường thô (vẫn chạy, chỉ chậm).

Quy ước đặt tên và nhãn

Một cách gọi cho mỗi thứ. Phiên làm việc sau tra bảng này trước khi đặt tên mới.

THỨ	TÊN CHUẨN
Database Turso	`intel-rollup`
Bảng rollup	`ru_cust_day`, `ru_sku_day`
Bản sao dim	`dm_source`, `dm_customer`, `dm_sku`, `dm_staff`
Bảng điều khiển	`ru_meta`, `ru_runlog`
Thư mục mã	`web/src/lib/rollup/` (`schema`, `builder`, `reader`, `router`, `reconcile`)
Route	`/api/cron/rollup-refresh`, `/api/analytics/rollup-health`
Tiền tố khóa cache	`ru:` + `data_version`
Cờ bật tab	`app_settings` khóa `rollup_routes`: `{ "route": "off|shadow|on" }`

CỘT / NHÃN	Ý NGHĨA
`basis`	`ful` = theo ngày giao (Fulfillment), `sal` = theo ngày tạo đơn (Created)
`day`	chuỗi `YYYY-MM-DD`, đúng định dạng cột ngày gốc
`ship`	1 nếu SKU là `SHIPPINGFEE0`
`revenue_vnd`, `cogs_vnd`, `gp_vnd`	Doanh thu, Giá vốn, Gross Profit (GP). Ở `sal`, `cogs` và `gp` bằng 0 như hiện nay
`qty`, `lines`, `orders`	số lượng; số dòng; số đơn khác nhau trong ngày đó
CM1, GPM %, CM1 %	KHÔNG lưu trong rollup: tính lúc đọc từ GP trừ chi phí (chi phí vẫn ở Supabase/Turso như cũ). Nhãn giao diện giữ `GP` và `CM1`; khóa dữ liệu `gpm2` giữ nguyên
Tier	Strategic / VIP / Gold / Silver: phân hạng lúc đọc bằng hàm hiện có (`makeClassifyTier`), không lưu

Khi dữ liệu bị sửa, không chỉ thêm mới

Ví dụ tên đổi từ A thành B. Mỗi loại thay đổi có một đường cập nhật riêng và một độ trễ tối đa.

LOẠI THAY ĐỔI	XỬ LÝ	ĐỘ TRỄ TỐI ĐA
Đổi nhãn trong dim (tên kênh, khách, SKU: A thành B)	Rollup lưu mã nên không dựng lại. `dm_*` được làm mới sau ETL dim; toàn bộ lịch sử hiện tên mới (đúng như hôm nay vì nối dim hiện tại)	sau lần làm mới kế tiếp, ≤ 1 giờ
Sửa giá trị trong fact (COGS, đơn hủy, xóa dòng)	Băm theo ngày khác nên xóa cả ngày rồi chèn lại, dòng bị xóa cũng mất theo	≤ 1 giờ trong cửa sổ 45 ngày; ngoài cửa sổ ≤ 24 giờ (đối chiếu từng tháng hằng đêm)
Đổi mã (gộp khách X vào Y)	Như sửa giá trị vì mã nằm trong fact; `dm_customer` đồng bộ cả dòng bị xóa	như trên
Thuộc tính nướng vào rollup: nhóm vendor của SKU (`hk3_rev_vnd`), cờ `ship`	Điểm yếu duy nhất. Băm bảng ánh xạ SKU→nhóm vendor; đổi băm thì dựng lại số đo đó cho toàn lịch sử ngay lần làm mới sau, không chờ cuối tuần	≤ 1 giờ
Cấu hình ở Supabase (loại khách, tier, chi phí, target)	Áp lúc đọc và nằm trong khóa cache bằng băm cấu hình (như `exclHash` hiện nay)	ngay

Cache máy chủ

Khóa gồm `data_version` và băm cấu hình: bản mới là khóa mới, không còn phục vụ bản cũ (đường hiện tại còn trả bản cũ tới 6 giờ sau hết hạn). Bản sao `dm_*` và rollup ghi cùng một lần, tăng `data_version` ở bước cuối, nên không có trạng thái nửa mới nửa cũ.

Cache biên (CDN)

`phải sửa` Header hiện tại là `s-maxage=300, stale-while-revalidate=600` : tên cũ có thể còn thấy thêm 5 đến 15 phút theo URL. Route rollup chuyển sang `no-store` vì đã đủ nhanh để không cần cache biên.

Cache trình duyệt

Trang đang hiện số lần xem trước rồi thay bằng số mới (`stale-first`). Thêm `data_version` vào khóa đó để khi dữ liệu đổi, bản cũ không hiện thoáng qua.

Quy tắc bất biến

Vi phạm một điều thì dừng và hỏi.

1. Rollup chỉ chứa số CỘNG ĐƯỢC. Tỷ lệ (%, trung bình) và mọi thứ cần chi phí kênh luôn tính lúc đọc.
2. Rollup lưu MÃ, không lưu nhãn. Tên kênh, tên khách, vendor lấy từ `dm_*` lúc đọc.
3. Bộ lọc cấu hình được (danh sách loại khách, tier, kênh) áp lúc đọc, không nướng vào rollup, để đổi cấu hình không phải dựng lại.
4. Chỉ cron ghi Turso rollup; reader chỉ đọc. Không route nào của người dùng gọi `gohub_dw` ở đường bình thường.
5. Mỗi tab bật rollup phải qua shadow và đối chiếu khớp; luôn còn đường thô để rơi về.
6. Số liệu hiển thị không đổi so với trước; tab nào lệch thì đó là lỗi của rollup, không phải "cách tính mới".
7. Đổi định nghĩa số liệu (ví dụ cách tính CM1) làm ở MỘT chỗ dùng chung, không viết lại trong từng route.

Rủi ro và câu hỏi còn mở

Rủi ro đã nhận diện

`ETL sửa số cũ` COGS và dữ liệu cũ có thể bị cập nhật. Đối chiếu từng tháng hằng đêm là lưới bắt.

`Hạn mức ghi` Nếu băm đổi nhiều ngày mỗi giờ, ghi tăng nhanh. Có chốt chặn 70% và 90%.

`Độ trễ Turso` ~150ms mỗi truy vấn, gửi song song nên chấp nhận được; sẽ đo thật ở P0.

`Phụ thuộc đường thô` Tổ hợp lọc SKU cộng loại khách vẫn đi đường thô, chậm hơn.

Chưa biết, sẽ làm rõ ở P0/P1

Lịch chính xác và thời lượng của job ETL đơn tạo (`ETL_fact_sales…`); thời lượng mỗi job.

Danh sách đầy đủ số đo theo nhóm vendor (ngoài `hk3_rev_vnd`) mà Quarter Report và các tab khác cần.

Cột nào của `fact_*` có thể bị ETL sửa lại sau khi ghi (quyết định cửa sổ làm mới 45 ngày là đủ hay cần dài hơn).

Điều kiện `excludeInactiveCustomers` áp ở rollup khách như thế nào cho khớp tuyệt đối.